Plant Guardian

Aplicación móvil para el cuidado inteligente de plantas de exterior e interior

Informe inicial

01/03/2026

Fiorella Lucia Queirolo Chavez

Índice

1. Introducción....3
2. Definición del proyecto....3
2.1. Contexto....3
2.2. Objetivos....3
3. Estudio de mercado....4
3.1. Justificación estratégica....4
3.1.1. Priorización de funcionalidades....4
3.1.2. Mercado objetivo....5
3.1.3. MVP y escalabilidad....5
3.1. Matriz comparativa....5
3.2. Conclusiones del estudio....6
4. Planificación....6
4.1. Metodología SCRUM....6
4.2. Fases y duración....7
4.3. Diagrama de Gantt....7
4.4. Tareas y actividades....8
4.5. Diagrama WBS....9
5. Tecnología....10
5.1. Aplicaciones de desarrollo....10
5.2. Inteligencia artificial....11
6. Webgrafía....12

1. Introducción

Este documento constituye el plan inicial para el desarrollo de Plant Guardian, una aplicación móvil orientada al cuidado botánico doméstico mediante la integración de inteligencia artificial y dinámicas de gamificación.

El informe presenta la base estratégica del sistema, analizando el contexto, las necesidades y los objetivos principales del proyecto a partir de un estudio de mercado previo. Asimismo, se describen las fases de desarrollo planificadas bajo el marco de trabajo SCRUM, estructuradas mediante un diagrama de Gantt y un diagrama WBS.

2. Definición del proyecto

En esta sección se explica que es la aplicación Plant Guardian, identificando los objetivos que se buscan lograr y los posibles interesados en el sistema.

2.1. Contexto

La idea de este proyecto nace con el propósito de ayudar a aquellas personas que no cuentan con conocimientos suficientes sobre el cuidado de plantas, tanto de interior como de exterior. La falta de información puede provocar problemas como un riego incorrecto, una exposición inadecuada a la luz o la detección tardía de enfermedades, afectando negativamente a la salud y al desarrollo de las plantas [1].

En este contexto surge Plant Guardian, una aplicación móvil para Android que actúa como un asistente personalizado que, mediante reconocimiento visual y análisis de síntomas, permite a los usuarios gestionar la salud de su colección de plantas. Además, la plataforma integra una capa de gamificación que premia la constancia en las rutinas de cuidado.

Lo que distingue a Plant Guardian es su compromiso con la simplicidad. No sólo monitoriza, sino que guía al usuario, proporcionando soluciones reales y accesibles. Es una herramienta perfecta para quienes buscan resultados profesionales en su jardín personal sin complicaciones y de forma totalmente móvil.

2.2. Objetivos

La meta principal de este proyecto es el desarrollo de un **Producto Mínimo Viable (MVP)** [2] que consolide las funcionalidades esenciales de Plant Guardian, centradas en la identificación inteligente de plantas mediante visión artificial y en la generación de rutinas de cuidado personalizadas, integradas en una experiencia de usuario gamificada que fomente la constancia en el cuidado vegetal.

Como parte estructural del sistema, se busca implementar el diseño e implementación de una arquitectura cliente-servidor que garantice la gestión y persistencia de los datos de usuarios y plantas, permitiendo almacenar historiales de cuidado, rutinas personalizadas y la evolución del estado de cada planta. De este modo, el usuario podrá disponer de un seguimiento continuo y personalizado.

Teniendo en cuenta todo lo mencionado, se han definido los tres objetivos principales que se muestran en la siguiente tabla.

ID	Objetivo
OBJ-1	**Desarrollar un sistema inteligente de asistencia para el cuidado de plantas.** Implementar el reconocimiento automático de plantas, el diagnóstico preventivo de patologías vegetales y la creación de una rutina personalizada para cada planta mediante la integración de APIs externas.
OBJ-2	**Diseñar una interfaz gamificada centrada en la experiencia de usuario.** Crear una experiencia interactiva basada en recompensas e insignias que fomente el compromiso a largo plazo y promueva hábitos responsables en el cuidado de las plantas.
OBJ-3	**Implementar un repositorio de datos centralizado para el seguimiento y la personalización de cada planta.** Desarrollar un backend estructurado con una base de datos que permita almacenar las rutinas personalizadas y los datos de las plantas de cada usuario.

3. Estudio de mercado

El presente estudio de mercado tiene como finalidad justificar las decisiones aplicadas en el desarrollo de Plant Guardian, tanto en relación con la plataforma tecnológica seleccionada como al posicionamiento competitivo del producto.

3.1. Justificación estratégica

La decisión de desarrollar Plant Guardian de forma nativa para la plataforma Android no es arbitraria, ya que se basa en los siguientes tres aspectos: viabilidad técnica, alcance de mercado y escalabilidad futura.

3.1.1. Priorización de funcionalidades

Dado que el núcleo del proyecto es la integración de IA y visión por computador, se ha optado por un entorno de desarrollo ya utilizado y conocido: **Android Studio** [3]. Esta elección permite realizar un *benchmarking* técnico sobre las capacidades de integración de APIs de Google, facilitando la comparativa de rendimiento frente a otros entornos [4].

El uso de esta plataforma permite centrarse en la implementación de la lógica central de la aplicación para una gestión eficiente de recursos del dispositivo y una buena comunicación con las APIs externas. Con esto, se facilita la separación de responsabilidades entre la capa frontend, la lógica de la aplicación y la comunicación con el backend. Esta estructura favorece la mantenibilidad del sistema y reduce la complejidad técnica en la fase de desarrollo.

3.1.2. Mercado objetivo

Según datos de StatCounter Global Stats, Android se posiciona como el sistema operativo móvil líder tanto a nivel global como nacional, manteniéndose en éste último con un 75% ~ 80% de cuota de mercado, frente al 20 ~ 25% correspondiente a iOS [5].

Estos datos indican que el desarrollo inicial sobre el sistema Android maximiza el alcance potencial del MVP, permitiendo validar la propuesta de valor en el mercado móvil, sobre todo a nivel nacional.

3.1.3. MVP y escalabilidad

Tal y como se ha dicho anteriormente, el enfoque actual se centra en la creación de un MVP funcional. No obstante, la arquitectura del sistema consta de modularidad y separación de responsabilidades para que la lógica y los servicios de IA se gestionen desde un backend independiente del frontend.

Esta arquitectura cliente-servidor [6] permite:

- Escalar los servicios de IA de forma independiente.
- Incorporar nuevos clientes como, por ejemplo, una versión en iOS.
- Adaptar el frontend a frameworks multiplataforma como Flutter o React Native sin necesidad de modificar la lógica central del sistema.

De este modo, se contempla la posibilidad de que el sistema llegue a formar parte de otras plataformas más allá de Android.

3.1. Matriz comparativa

Para seleccionar las aplicaciones a comparar, se han aplicado los siguientes criterios basados en un análisis comparativo, el cual permite identificar las mejores prácticas y brechas de valor en el mercado actual [7]:

- **Relevancia:** Aplicaciones con al menos más de 10 mil descargas y un rating superior a 4 estrellas.
- **Funcionalidades:** Aplicaciones que tengan funcionalidades similares y que tengan el mismo objetivo del cuidado de plantas.

Funcionalidad	Plant Parent	Plant Master	Planti
Reconocimiento visual (IA)	✅	✅	✅
Diagnóstico de síntomas	✅	✅	❌
Rutinas personalizadas (IA)	✅	✅	✅
Influencia de los datos meteorológicos	❌	❌	❌
Historial de plantas y cuidados	✅	❌	❌
Gamificación y recompensas	❌	❌	❌

3.2. Conclusiones del estudio

A partir del análisis anterior realizado sobre Plant Master, Plant Parent y Planti, se observa que el mercado actual presenta un grado elevado en lo relativo al reconocimiento visual de especies vegetales mediante IA, así como en la incorporación de diagnósticos básicos de síntomas y patologías. Estas funcionalidades se podrían considerar como un estándar dentro del sector del cuidado de plantas.

Sin embargo, en el análisis también se pueden observar ciertas limitaciones comunes:

- La personalización de rutinas no siempre está automatizada mediante IA, sino que en algunos casos depende de la configuración manual del usuario o de cuidados básicos.
- La integración de datos locales, como la información meteorológica local, no se encuentra generalizada entre las aplicaciones vistas.
- Ninguna de las aplicaciones analizadas incorpora mecanismos de gamificación orientados a reforzar la constancia y el compromiso del usuario con el cuidado de sus plantas.

En este contexto, Plant Guardian se centra en reforzar las siguientes funcionalidades:

- Generación automatizada de rutinas personalizadas mediante IA.
- Consideración de variables ambientales externas.
- Un sistema de incentivos basado en dinámicas de recompensa.

De este modo, el proyecto no compite por la capacidad técnica, sino por términos de experiencia de usuario y motivación mediante la gamificación, lo que lo diferencia a comparación de las otras aplicaciones.

4. Planificación

Antes de empezar el proyecto, primero se ha realizado una planificación general de las tareas a realizar, juntamente definiendo la metodología que se usará durante los próximos meses y las fases que abordará el proyecto.

4.1. Metodología SCRUM

Para este proyecto se ha optado por utilizar la metodología Scrum [8], combinando los roles de *product owner*, *scrum master* y desarrollador, ya que este proyecto está pensado para realizarse individualmente. Bajo este enfoque, las tareas a realizar se organizan según el inicio de cada sprint.

La planificación se ha estructurado siguiendo los hitos de entrega establecidos para el TFG. El objetivo es que cada fase agrupe las tareas correspondientes, manteniendo la flexibilidad de Scrum, la cual permite integrar tareas pendientes de ciclos anteriores o adelantar requerimientos futuros según las necesidades del desarrollo.

Para el control y el seguimiento del proyecto, se ha optado por utilizar la plataforma **Trello**, la cual se ha configurado para que siga el flujo de trabajo de Scrum [9].

4.2. Fases y duración

El desarrollo se basa en cuatro fases consecutivas, diseñadas para cubrir todo el ciclo de vida del proyecto, desde su concepción hasta la entrega final. Al tratarse de un proyecto individual, la carga de trabajo se ha distribuido para garantizar la obtención de una primera versión funcional que incluya los objetivos principales.

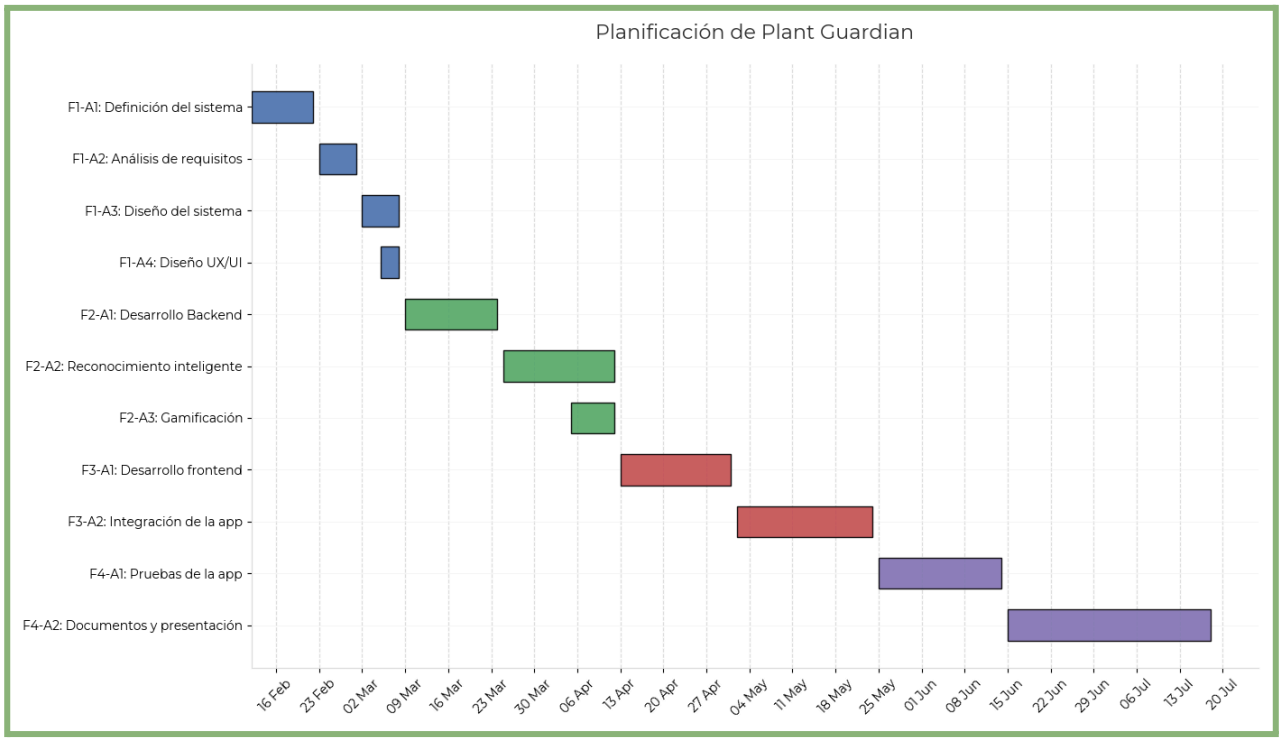
La prioridad será la implementación de las funcionalidades clave, es decir, **completar los tres objetivos y todos los requisitos esenciales**. Con esto en mente, las fases para este proyecto se han distribuido de la siguiente manera.

Fase	Descripción	Inicio	Fin
Fase 1	Definición, análisis y diseño	12/02	08/03
Fase 2	Desarrollo de la BD, APIs y lógica	09/03	12/04
Fase 3	Desarrollo del frontend e integración de la app	13/04	24/05
Fase 4	Finalización de la app	25/05	18/07

4.3. Diagrama de Gantt

El desarrollo del proyecto se ha organizado siguiendo un diagrama de Gantt, con un calendario comprendido entre febrero y julio de 2026.

La planificación se estructura en las cuatro fases mencionadas previamente, cada una de las cuales incluye un conjunto de actividades. En el siguiente apartado se describirán los paquetes de trabajo correspondientes a cada actividad.



Cabe decir que, dada la naturaleza académica y los plazos fijos de entrega del TFG, se ha creado este diagrama de Gantt para definir la duración de los hitos de cada fase. No obstante, el desarrollo técnico se abordará bajo una filosofía ágil (tal y como se ha dicho, será SCRUM), utilizando los tableros de gestión de tareas para permitir la adaptabilidad dentro de cada fase.

4.4. Tareas y actividades

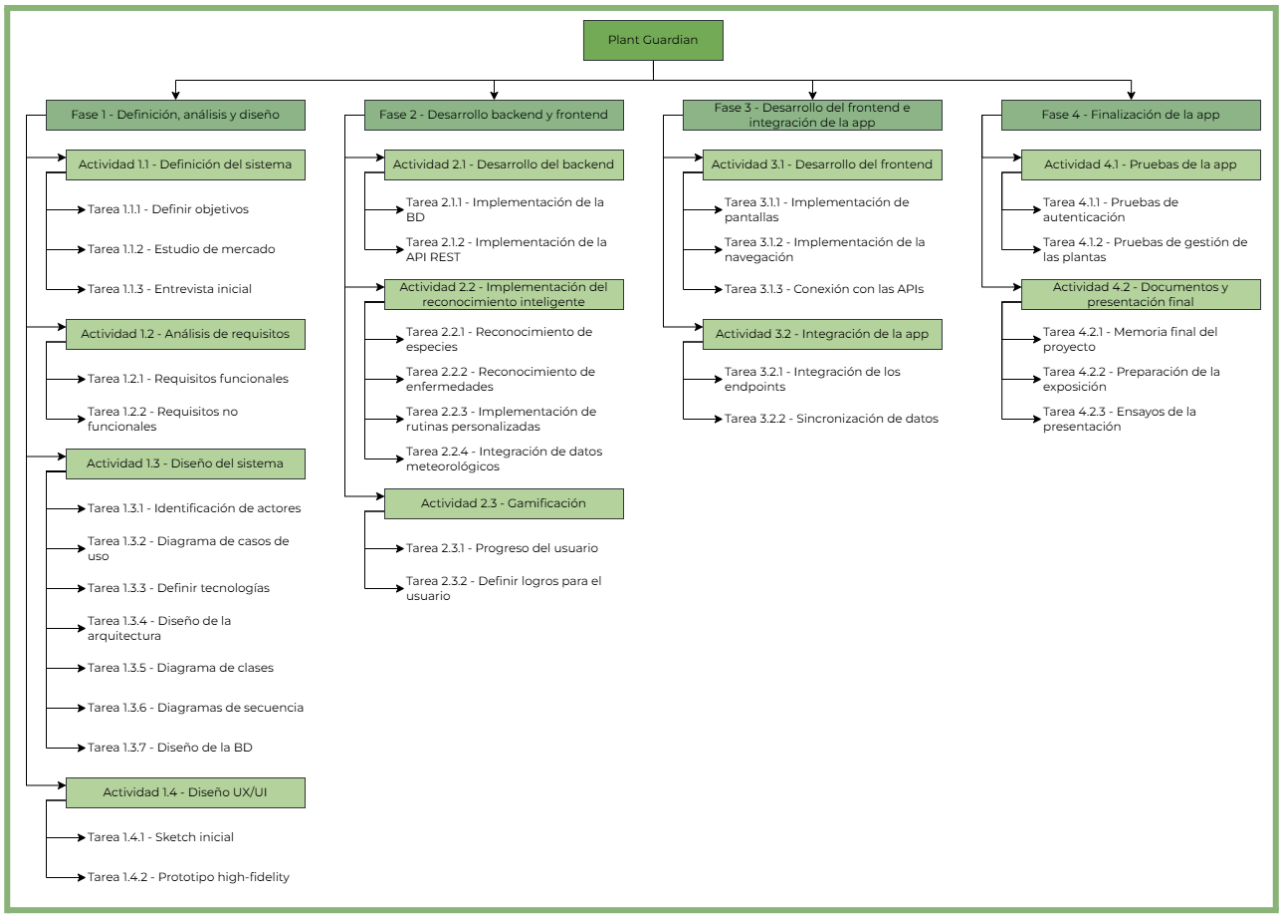
A continuación, se desglosan las tareas y actividades para cada fase. Estas tareas son las que se especificarán en el tablero de Trello.

Fase 1 - Definición, análisis y diseño	
	Actividad 1.1 - Definición del sistema
	Tarea 1.1.1 - Definir objetivos
	Tarea 1.1.2 - Estudio de mercado
	Tarea 1.1.3 - Entrevistas iniciales
	Actividad 1.2 - Análisis de requisitos
	Tarea 1.2.1 - Requisitos funcionales
	Tarea 1.2.2 - Requisitos no funcionales
	Actividad 1.3 - Diseño del sistema
	Tarea 1.3.1 - Identificación de actores
	Tarea 1.3.2 - Diagrama de casos de uso
	Tarea 1.3.3 - Definir tecnologías
	Tarea 1.3.4 - Diseño de la arquitectura
	Tarea 1.3.5 - Diagrama de clases
	Tarea 1.3.6 - Diagramas de secuencia
	Tarea 1.3.7 - Diseño de la BD
	Actividad 1.4 - Diseño UX/UI
	Tarea 1.4.1 - Sketch inicial
	Tarea 1.4.2 - Prototipo high-fidelity
Fase 2 - Desarrollo de la BD, APIs y lógica	
	Actividad 2.1 - Desarrollo del backend
	Tarea 2.1.1 - Implementación de la BD

	Tarea 2.1.2 - Implementación de la API REST
Actividad 2.2 - Implementación del reconocimiento inteligente	
	Tarea 2.2.1 - Reconocimiento de especies
	Tarea 2.2.2 - Reconocimiento de enfermedades
	Tarea 2.2.3 - Implementación de rutinas personalizadas
	Tarea 2.2.4 - Integración de datos meteorológicos
Actividad 2.3 - Gamificación	
	Tarea 2.3.1 - Progreso del usuario
	Tarea 2.3.2 - Definir logros para el usuario
Fase 3 - Desarrollo del frontend e integración de la app	
Actividad 3.1 - Desarrollo del frontend	
	Tarea 3.1.1 - Implementación de pantallas
	Tarea 3.1.2 - Implementación de la navegación
	Tarea 3.1.3 - Conexión con las APIs
Actividad 3.2 - Integración de la app	
	Tarea 3.2.1 - Integración de los endpoints
	Tarea 3.2.2 - Sincronización de datos
Fase 4 - Finalización de la app	
Actividad 4.1 - Pruebas de la app	
	Tarea 4.1.1 - Pruebas de autenticación
	Tarea 4.1.2 - Pruebas de gestión de las plantas
Actividad 4.2 - Documentos y presentación final	
	Tarea 4.2.1 - Memoria final del proyecto
	Tarea 4.2.2 - Preparación de la exposición
	Tarea 4.2.3 - Ensayos de la presentación

4.5. Diagrama WBS

El diagrama *Work Breakdown Structure* (WBS) [10] incluye las fases, tareas y actividades explicadas anteriormente de manera jerárquica y visual. Todos los paquetes de trabajo están pensados para cumplir los requisitos del sistema y del proyecto.



5. Tecnología

Para el desarrollo de la aplicación se emplearán las siguientes herramientas y tecnologías.

5.1. Aplicaciones de desarrollo

En cuanto a la infraestructura y las herramientas de desarrollo, se utilizará:

- **GitHub:** Plataforma de control de versiones y gestión de código fuente, que permite mantener un historial de cambios y colaborar de manera eficiente.
- **Visual Studio Code:** Entorno de desarrollo utilizado para la implementación del backend en Python y la integración de llamadas a APIs externas.
- **Android Studio:** Entorno de desarrollo para la creación del frontend móvil, utilizando XML para la interfaz de usuario y Kotlin para las llamadas al backend.
- **Figma:** Aplicación para desarrollar el prototipo *high-fidelity* y entender el flujo del usuario al utilizar la aplicación.

Por otro lado, para la inteligencia del sistema se prevé las siguientes APIs externas:

- **Pl@ntNet:** Motor de búsqueda visual para identificación de especies.
- **Trefle:** Repositorio de datos botánicos y requerimientos de cultivo.
- **OpenWeatherMap:** Consulta de condiciones climáticas locales.
- **Gemini:** Herramienta de IA para el análisis de la planta y la creación de las rutinas.

Cabe decir que se contempla como **alternativa** el uso de Gemini para sustituir a Pl@ntNet y Trefle, ya que su capacidad de análisis permite realizar tanto la identificación de la planta como la obtención de información botánica. Por lo tanto, esta decisión se determinará durante el desarrollo del proyecto y, por el momento, se integrarán dichas APIs en la arquitectura del sistema.

Por último, el núcleo de la arquitectura lo conformará **SupaBase**, una plataforma BaaS utilizada para la gestión de la base de datos relacional con PostgreSQL, la autenticación de usuarios y el almacenamiento de imágenes.

5.2. Inteligencia artificial

Para este proyecto, se utilizan las herramientas de inteligencia artificial **ChatGPT** y **Gemini** como recursos de apoyo durante el desarrollo. El uso de cada una se basa en lo siguientes puntos:

- **Consultas técnicas:** Soporte en la toma de decisiones estratégicas. Por ejemplo, en la selección y comparativa técnica de APIs.
- **Optimización:** Generación de estructuras de código base, la resolución de errores de compilación y la optimización de algoritmos.
- **Supervisión:** Todo fragmento de código o sugerencia es analizado y adaptado para garantizar que se ajusta a los requisitos del proyecto, asegurando la calidad, la seguridad y la integración del sistema.

En resumen, la asistencia de la IA incrementa la eficiencia operativa. Con esta ayuda, se busca que el desarrollo individual durante los próximos meses sea más ágil y permita completar los objetivos planificados.

6. Webgrafía

[1] *¿No sabes por qué tu planta está enferma? Los problemas más comunes, sus síntomas y cómo curarlas*. (n.d.). Retrieved February 13, 2026, from https://viverosgonzalez.es/es/blog-viveros-gonzalez/68-planta-enferma

[2] *Significado de MVP - Qué es un Producto Mínimo Viable (MVP) | Mailchimp*. (n.d.). Retrieved February 13, 2026, from https://mailchimp.com/es/resources/minimum-viable-product/

[3] *Introducción a Android Studio | Android Developers*. (n.d.). Retrieved February 13, 2026, from https://developer.android.com/studio/intro?hl=es-419

[4] *Benchmarking: qué es, ejemplos y cómo hacer uno | Blog UE*. (n.d.). Retrieved February 14, 2026, from https://universidadeuropea.com/blog/benchmarking-que-es/

[5] *Operating System Market Share Worldwide | Statcounter Global Stats*. (n.d.). Retrieved February 14, 2026, from https://gs.statcounter.com/os-market-share

[6] *Arquitectura Cliente-Servidor*. (n.d.). Retrieved February 13, 2026, from https://reactiveprogramming.io/blog/es/estilos-arquitectonicos/cliente-servidor

[7] *Análisis Comparativo: Guía Práctica para Profesionales - Zinklar*. (n.d.). Retrieved February 14, 2026, from https://zinklar.com/es/blog/que-es-un-analisis-comparativo-una-guia-practica-para-profesionales/

[8] *¿Qué es scrum? Una guía para el marco de trabajo ágil | Atlassian*. (n.d.). Retrieved February 13, 2026, from https://www.atlassian.com/es/agile/scrum

[9] *Gestiona proyectos SCRUM con TRELLO | Bring Tips - YouTube*. (n.d.). Retrieved February 16, 2026, from https://www.youtube.com/watch?v=UldlLhT3Ctc

[10] *¿Qué es una estructura de desglose de trabajo? | Miro*. (n.d.). Retrieved February 17, 2026, from https://miro.com/es/gestion-proyectos/que-es-wbs/